

CS541 Project 4: Shadows

Synopsis

Implement shadows using a two-pass shadow map algorithm. Supply solutions for the shadow-acne produced by the basic algorithm.

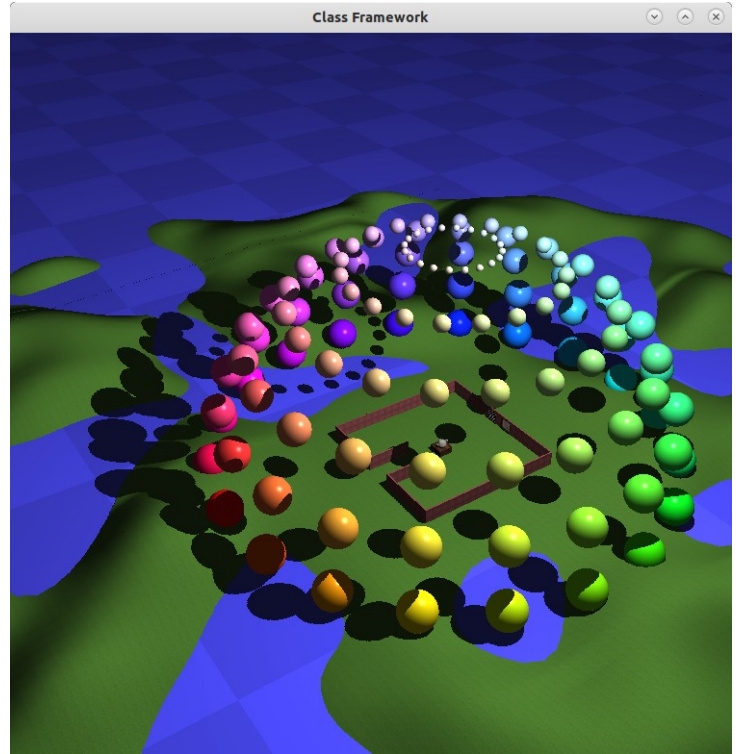
Instructions

For a shadow-map friendly test scene, turn on the hemisphere-of-spheres with a **true** in this line at the top of **scene.cpp**:

```
const bool showSpheres = true;
```

The basic algorithm is described on the next page. One decision that needs to be made during that algorithm is to decide what to do with points that are outside the range of the shadow map. Your choices are:

- Light all out-of-range points, making sure the range of the shadow map is sufficient to cover all objects which can cast shadows.
- Shadow out-of-range points, producing a square spot light effect
- Shadow points outside the central circular portion of the shadow map, producing a circular spot-light, possibly with a soft variable fade out near the outer edge.



Once you get the basic shadow-mapping working, you will notice the so called shadow-acne. (See the image below.) Remove the shadow-acne by implementing both of these solutions:

- **Shadow depth offset:** Instead of comparing two depths for equality (always folly with floating point numbers), add a small offset to compensate for roundoff and discretization errors.
- **Front face culling while generating the shadow map:** Enable front-face culling (during the first pass) by surrounding the drawing calls with glEnable/glDisable:

```
glEnable(GL_CULL_FACE)  
glCullFace(GL_FRONT)  
<<draw geometry here>>>  
glDisable(GL_CULL_FACE)
```

Grading Basis

Grading will be on a 100 point basis, with the following distribution:

- **Accuracy:** 70%, judgment based on looking at the results on the screen.
- **Removing acne:** 20%, based mostly evidence in your project report of what you tried and your judgment of how well it worked.
- **Code:** 10%, judgment based on looking at the code for proper implementation and reasonable commenting practices.

Shadow Map Algorithm

Initialization:

Create an FBO of sufficient size, say 1K by 1K. (See **fbo.h** and **fbo.cpp** for an encapsulation of the FBO operations.)

Create a new shader for shadow map creation from two new files, perhaps named **shadow.frag** and **shadow.vert**. (See **shader.h** and **shader.cpp** for the encapsulation, and **scene.cpp** for an example.)

Pass 1: Render from light's POV, to a Frame Buffer Object (FBO)

DrawScene:

Use the shadow shader and **Bind** the FBO

Create transformations from the light's POV and send to the shader

ModelMatrix: built by application during scene drawing

ViewMatrix: $V_L = \text{LookAt}(\text{LightPos}, \text{LightDir}, \text{UpDir})$

ProjectionMatrix: $P_L = \text{Perspective}(\dots)$

Draw all the geometry

Unbind the FBO and **Unuse** the shader.

Vertex processor:

Feed the graphics pipeline (as always):

$\text{gl_Position} = \text{ProjectionMatrix} * \text{ViewMatrix} * \text{ModelMatrix} * \text{vertex}$

Compute the vertex depth from the light's POV and output:

out vec4 position

position = gl_Position (position.w is light's depth)

Fragment processor:

Write the depth (that is, position.w) to the FBO

in vec4 position

FragColor = position (only the w coordinate will be used)

Pass 2: Render from eye's POV to screen:

Application

Create transformations from the Eye's POV and send to the shader

ModelMatrix: built by application during scene drawing

ViewMatrix: $V =$ built by application before scene drawing

ProjectionMatrix: $P =$ built by application before scene drawing

ShadowMatrix: $B P_L V_L$ Used to transform position in world coordinates to shadow-texture coordinates
where $B = \text{Translate}(\frac{1}{2}, \frac{1}{2}, \frac{1}{2}) * \text{Scale}(\frac{1}{2}, \frac{1}{2}, \frac{1}{2})$

Vertex processor:

Feed the graphics pipeline (as always):

$\text{gl_Position} = \text{ProjectionMatrix} * \text{ViewMatrix} * \text{ModelMatrix} * \text{vertex}$

Compute the vertex position in the light's POV and output

out vec4 shadowCoord;

shadowCoord = ShadowMatrix * ModelMatrix * vertex

Fragment shader:

Project the interpolated pixel to the shadow-map:

vec2 shadowIndex = shadowCoord.xy/shadowCoord.w

Check if shadowIndex is in range

shadowCoord.w > 0 and shadowIndex.x (and .y) in 0..1 range

Light-depth = texture2d(shadowMap, shadowIndex).w

Pixel-depth = shadowCoord.w

Pixel is in shadow if: Pixel-depth > Light-depth

Color pixel accordingly: ambient only (if in shadow),

ambient+diffuse+specular (if not in shadow).

Working with textures

In a shader: Declare a texture variable as
uniform sampler2D shadowMap;

and access/read the texture via

texture(shadowMap, uv);

where **uv** is a **vec2** with values in the range $[0...1]$.

In CPU code: Activate a texture unit, load the texture into it, and tell the sampler2D variable which texture unit you chose. The texture's ID for this project is found in the FBO object, and the texture unit can be any of (at least) 16 texture units 0 through 15. Beware that the framework drawing code already uses texture units 0 and 1, so avoid using them.

```
glActiveTexture(GL_TEXTURE2)           // Activate texture unit 2
```

```
glBindTexture(GL_TEXTURE_2D, texID) // Load texture into it
```

```
int loc = glGetUniformLocation(shaderId, "shadowMap")
```

```
glUniform1i(loc, 2)                   // Tell shader texture is in unit 2
```

Texture debugging aid

To view a full-screen version of any texture, replace this at the top of your fragment shader:

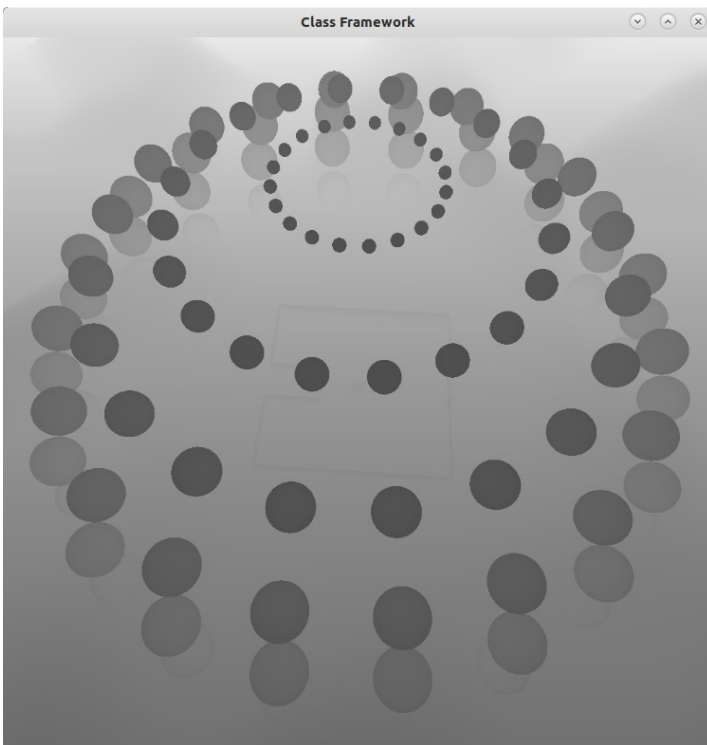
```
uniform sampler2D shadowMap;
```

```
vec2 uv = gl_FragCoord.xy/vec2(750,750); // (or whatever screen size)
```

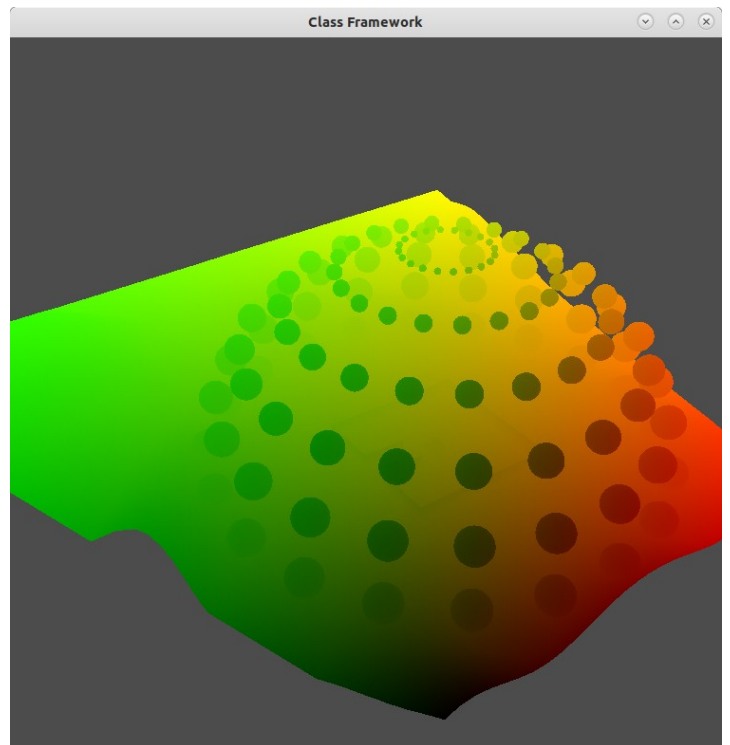
```
FragColor = vec3(texture(shadowMap, uv).w/100.0); // or similar
```

```
return; // which disables all further code in the shader.
```

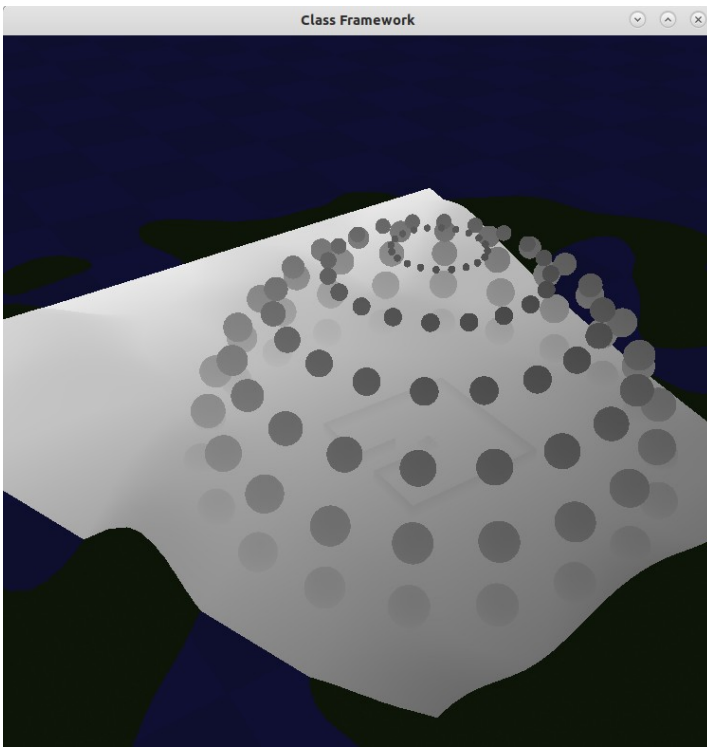
Some images for debugging purposes.



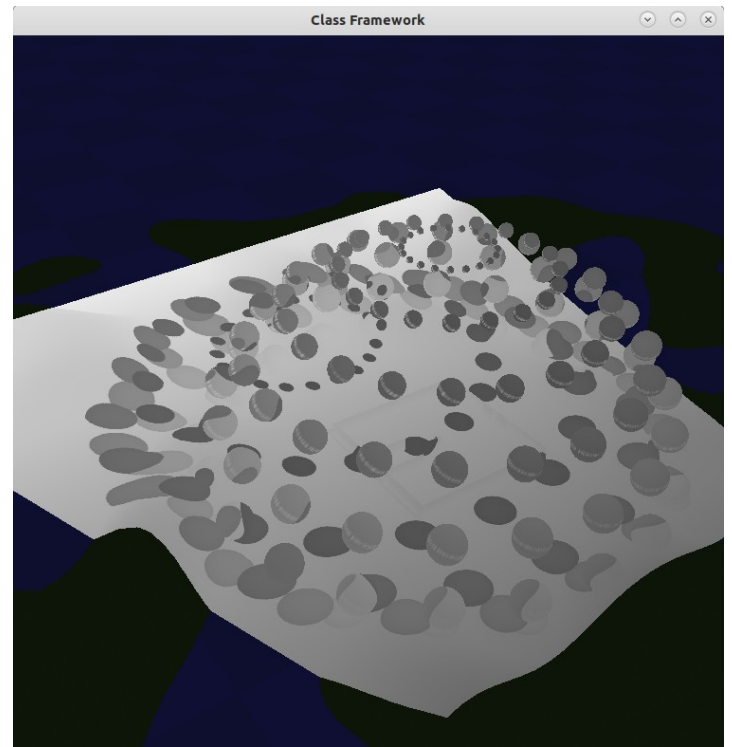
The depth-map divided by 100 and drawn as a gray-scale image.



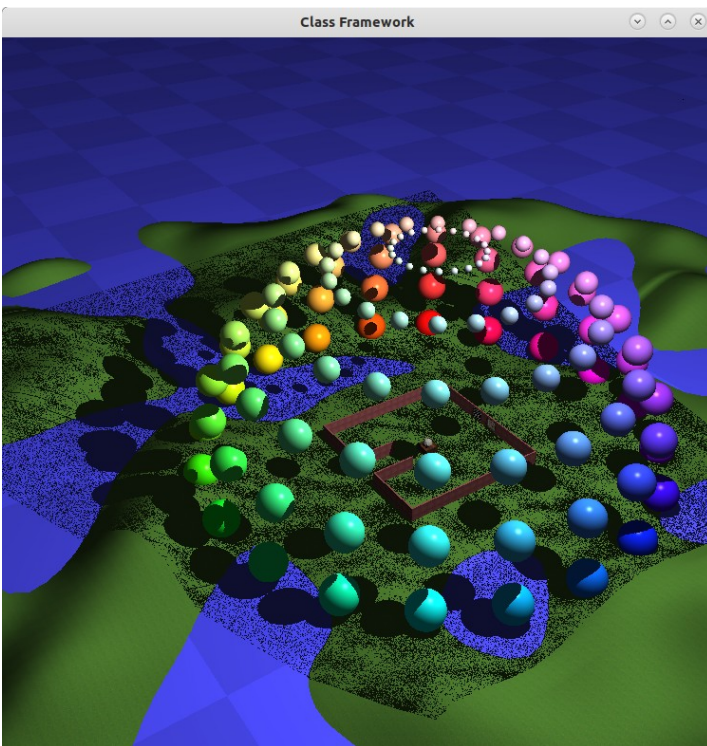
The shadowIndex vec2 drawn in the R&G channels for points inside the shadow frustum and gray for points outside.



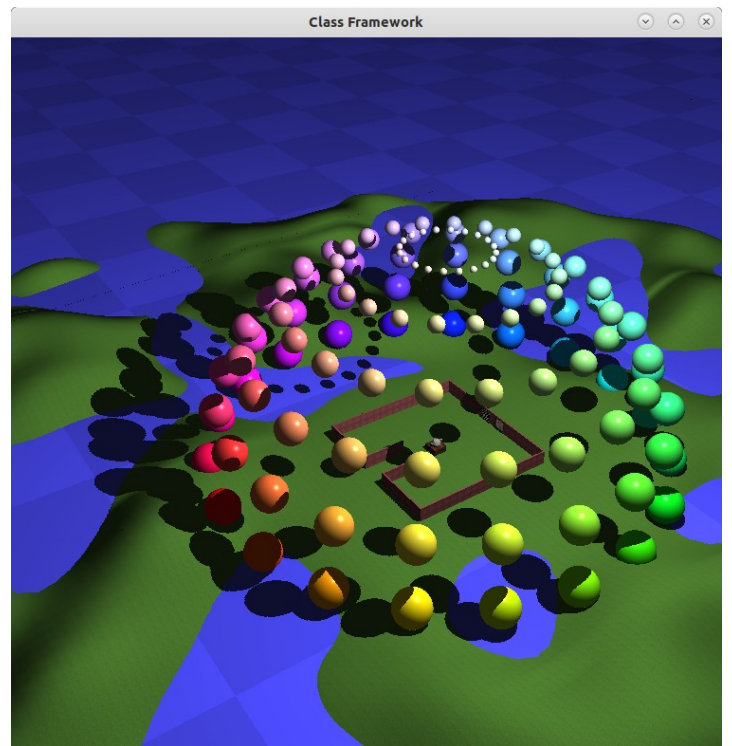
The pixel-depth, divided by 100 and drawn as a gray-scale image.



The light-depth, divided by 100 and drawn as a gray-scale image.



The final algorithm without shadow-acne solutions.



The final algorithm with shadow-acne solutions.

LookAt transformation

We want

- the eye (at E)
- to be looking at the LookAt point C .
- In addition we want the up vector U to project to the screen's Y axis.

Compute:

$$V = (C - E).normalized()$$

$$A = (V \times U).normalized()$$

$$B = A \times V$$

These form an orthonormal frame, and so can form the rows or columns of a rotation matrix.

Step 1: translate to origin:

$$T(-E)$$

Step 2: rotate with

$$R = \begin{bmatrix} A_x & A_y & A_z & 0 \\ B_x & B_y & B_z & 0 \\ -V_x & -V_y & -V_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R A^T = [1, 0, 0]^T$$

Note that: $R B^T = [0, 1, 0]^T$

$$R V^T = [0, 0, 1]^T$$

Final answer:

$$\text{LookAt matrix} = R * T(-E)$$

$$= \begin{bmatrix} A_x & A_y & A_z & -A \cdot E \\ B_x & B_y & B_z & -B \cdot E \\ -V_x & -V_y & -V_z & V \cdot E \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The consistent sign change in the 3rd row is the result of wanting the viewing transformation to look along the -Z axis.